

Classification — Project Recipe Card

End-to-end recipe · model toolbox · metrics · CV · class imbalance · threshold tuning · calibration · deployment. Deep companion: classification_cheatsheet.md + classification.ipynb

◆ Project recipe (12 steps)

- 1. **Frame** — binary / multi-class / multi-label; success metric tied to business action
- 2. **Stratified split FIRST** — train/val/test before EDA (no leakage)
- 3. **EDA + label audit** — class balance, label noise, leakage candidates
- 4. **Feature pipeline** — ColumnTransformer → impute → encode → scale
- 5. **Baseline** — DummyClassifier (most_frequent) + LogisticRegression
- 6. **Model toolbox** — LR → RF → GBM, climb the ladder
- 7. **CV with StratifiedKFold** — preserve class ratio in each fold
- 8. **Tune** — Optuna TPE (≥3 hp) or GridSearchCV
- 9. **Calibrate** — CalibratedClassifierCV if you need P(y=1) to be a true probability
- 10. **Threshold** — tune on VAL using precision-recall curve (NEVER on test)
- 11. **Test ONCE** — final eval; report confusion matrix + main metric
- 12. **Deploy** — pickle pipeline + log threshold + drift monitor

Universal skeleton: Pipeline([("prep", ColumnTransformer(...)), ("model", LogisticRegression(class_weight="balanced"))]) → tune → calibrate → threshold → ship.

◆ Starter skeleton · A: imports + split

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (roc_auc_score,
                             average_precision_score,
                             f1_score,
                             classification_report, confusion_matrix,
                             precision_recall_curve)

import matplotlib.pyplot as plt
np.random.seed(42)

# 1. Stratified split FIRST (3-way: train / val / test)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                         stratify=y,
                                         random_state=42)
X_tr, X_va, y_tr, y_va = train_test_split(X_tr, y_tr,
                                         test_size=0.2,
                                         stratify=y_tr,
                                         random_state=42)
# 60/20/20 – val drives calibration + threshold tuning
```

◆ Starter skeleton · B: pipeline + tune + test

```
# 2. Build pipeline
num = ["age", "income"]; cat = ["region", "segment"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())])), num),
    ("cat", OneHotEncoder(handle_unknown="ignore",
                          sparse_output=False), cat),
])
pipe = Pipeline([("prep", prep),
                 ("model",
                  LogisticRegression(class_weight="balanced",
                                     max_iter=1000,
                                     random_state=42))])

# 3. CV + grid
grid = {"model__C": [0.01, 0.1, 1, 10, 100]}
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
gs = GridSearchCV(pipe, grid, cv=skf, scoring="roc_auc",
                  n_jobs=-1)
gs.fit(X_tr, y_tr)

# 4. Test ONCE
p = gs.predict_proba(X_te)[: , 1]
print(f"ROC-AUC: {roc_auc_score(y_te, p):.3f}")
print(f"PR-AUC : {average_precision_score(y_te, p):.3f}")
```

● Frame + label audit

Frame checklist

- **Task:** binary / multi-class (k mutually exclusive) / multi-label (k independent)
- **Output:** hard label OR probability OR ranking
- **Cost asymmetry:** FP cost vs FN cost (drives threshold)
- **Class balance:** balanced (acc OK) vs imbalanced (PR-AUC / F1)
- **Label source:** hand-labeled / programmatic / proxy?

Label audit

```
y.value_counts(normalize=True) # class balance
# Multi-label:
y_df.sum(axis=0); y_df.sum(axis=1).hist() # label cardinality
# Label noise detection:
from cleanlab.classification import CleanLearning
cl = CleanLearning(pipe).fit(X_tr, y_tr)
noisy = cl.find_label_issues(X_tr, y_tr)
```

Imbalance levels

Minority %	Strategy
30-50%	nothing special; accuracy OK
5-30%	class_weight="balanced"; report F1/PR-AUC
1-5%	+ threshold tune + SMOTE try
<1%	treat as anomaly; PR-AUC primary; conformal prediction

Leakage candidates

Drop before split: any feature collected AFTER the prediction time, ID columns leaking class, post-event aggregates, target encoded outside CV.

● Feature pipeline

ColumnTransformer (canonical)

```
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())])), num_cols),
    ("cat", Pipeline([("imp",
                     SimpleImputer(strategy="most_frequent")),
                     ("oh",
                     OneHotEncoder(handle_unknown="ignore",
                                     sparse_output=False))])), cat_cols),
    ("txt", TfidfVectorizer(max_features=5000, "text_col"),
    ], remainder="drop", verbose feature_names_out=False)
prep.set_output(transform="pandas")
```

Encoding categoricals

Encoder	When
OneHot	low cardinality (<50); linear/NN
OrdinalEncoder	tree-based only (ordering won't confuse them)
TargetEncoder (CV-safe)	high cardinality (>50); leakage if not inside CV
CountEncoder	simple, fast, no leakage risk
Embeddings	NN only; very high cardinality (10k+)

sklearn 1.3+ has CV-safe TargetEncoder:
from sklearn.preprocessing import TargetEncoder
te = TargetEncoder(cv=5, random_state=42, smoothe="auto")

Text features

```
from sklearn.feature_extraction.text import TfidfVectorizer
TfidfVectorizer(max_features=10_000, ngram_range=(1,2),
                sublinear_tf=True, min_df=2)
# Modern: sentence-transformers for embedding-based features
```

Handling imbalance (data-level)

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
# Important: imblearn Pipeline so resamplers see only train
```

```
from sklearn.metrics import make_scorer
COST = {"FP": 1.0, "FN": 5.0} # FN 5x costlier
(fraud, disease, churn)
def custom_cost(y_true, y_pred):
    tn = ((y_pred==0)&(y_true==0)).sum(); fp =
    ((y_pred==1)&(y_true==0)).sum()
    fn = ((y_pred==0)&(y_true==1)).sum(); tp =
    ((y_pred==1)&(y_true==1)).sum()
    return -(COST["FP"]*fp + COST["FN"]*fn) # negative –
sklearn maximises
scorer = make_scorer(custom_cost, greater_is_better=True)
```

SMOTE on numeric features only; use SMOTENC for mixed.
Cost-sensitive learning (asymmetric FP/FN)

```
from sklearn.metrics import make_scorer
COST = {"FP": 1.0, "FN": 5.0} # FN 5x costlier
(fraud, disease, churn)
def custom_cost(y_true, y_pred):
    tn = ((y_pred==0)&(y_true==0)).sum(); fp =
    ((y_pred==1)&(y_true==0)).sum()
    fn = ((y_pred==0)&(y_true==1)).sum(); tp =
    ((y_pred==1)&(y_true==1)).sum()
    return -(COST["FP"]*fp + COST["FN"]*fn) # negative –
sklearn maximises
scorer = make_scorer(custom_cost, greater_is_better=True)

# Use as scoring in GridSearchCV / cross_val_score:
gs = GridSearchCV(pipe, grid, cv=skf, scoring=scorer)
# Or apply class_weight to bake into loss:
LogisticRegression(class_weight={0: 1, 1: 5}) # explicit
numeric weights
# Or per-sample weights:
sample_weight = np.where(y_tr == 1, 5.0, 1.0)
pipe.fit(X_tr, y_tr, model__sample_weight=sample_weight)
```

▲ Model toolbox (climb the ladder)

1 · DummyClassifier

```
from sklearn.dummy import DummyClassifier
DummyClassifier(strategy="most_frequent").fit(X_tr,
y_tr).score(X_te, y_te)
# "stratified" for ROC-AUC = 0.5 baseline

2 · Logistic Regression (canonical baseline)



```
from sklearn.linear_model import LogisticRegression
LogisticRegression(
 C=1.0, # inverse regularisation
 (high C = less reg)
 penalty="l2", # "l1" / "elasticnet" /
 None
 class_weight="balanced", # auto-balance from
 frequencies
 solver="lbfgs", # "saga" for L1/elasticnet
 max_iter=1000,
 random_state=42,
).fit(X_tr, y_tr)
```


```

Interpretable coeffs (after standardisation). Linear decision boundary.

3 · Linear SVM

```
from sklearn.svm import LinearSVC
LinearSVC(C=1.0, class_weight="balanced", random_state=42)
# No predict_proba – wrap in CalibratedClassifierCV if you need
probabilities

4 · Naive Bayes (text only)



```
from sklearn.naive_bayes import MultinomialNB, ComplementNB
MultinomialNB(alpha=1.0) # bag-of-words / TF-IDF
ComplementNB(alpha=1.0) # imbalanced text
```


```

5 · Random Forest

```
from sklearn.ensemble import RandomForestClassifier
RandomForestClassifier(
    n_estimators=500, max_depth=None,
    min_samples_leaf=5, class_weight="balanced",
    n_jobs=-1, random_state=42)
```

6 · Gradient boosting (THE workhorse)

```
import lightgbm as lgb
m = lgb.LGBMClassifier(
    n_estimators=2000, learning_rate=0.05,
    num_leaves=63, min_child_samples=20,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0,
```

```
class weight="balanced",      # OR scale_pos_weight=neg/
pos for binary
n_jobs=-1, random_state=42)
m.fit(X_tr, y_tr,
      eval_set=[(X_va, y_va)], eval_metric="auc",
      callbacks=[lgb.early_stopping(50)])
XGBoost: scale_pos_weight; CatBoost: handles cat natively.
```

Model picker

Situation	Start with
Need interpretable coefs	LogisticRegression(L1)
Text classification	TF-IDF + LinearSVC / NB
Tabular, >1k rows	LightGBM
Small N, calibrated	LR or RF + calibration
Multi-label	OneVsRest(LogReg or LGB)
Highly imbalanced (<1%)	GBM + threshold + PR-AUC primary

◆ Metrics

Confusion matrix vocabulary

	Pred 0	Pred 1
True 0	TN	FP (type-I)
True 1	FN (type-II)	TP

Precision = TP/(TP+FP) "of predicted positives, % correct"
Recall = TP/(TP+FN) "of true positives, % caught"
F1 = 2·P·R/(P+R) harmonic mean
Specificity = TN/(TN+FP) "true negative rate"

Threshold-free metrics

Metric	When
ROC-AUC	balanced classes; rank quality
PR-AUC (AvgPrec)	imbalanced — preferred over ROC
Log-loss	well-calibrated probabilities
Brier score	calibration + sharpness
Cohen's κ	vs chance-baseline (multi-class)

Canonical Python

```
from sklearn.metrics import (roc_auc_score,
                             average_precision_score,
                             log_loss, brier_score_loss,
                             f1_score,
                             classification_report,
                             confusion_matrix,
                             RocCurveDisplay,
                             ConfusionMatrixDisplay,
                             PrecisionRecallDisplay)
p = pipe.predict_proba(X_te)[: , 1]
print("ROC", roc_auc_score(y_te, p))
print("PR ", average_precision_score(y_te, p))
print("BS ", brier_score_loss(y_te, p))
RocCurveDisplay.from_predictions(y_te, p)
PrecisionRecallDisplay.from_predictions(y_te, p)
print(classification_report(y_te, (p > 0.5).astype(int)))
```

Multi-class

Average	When
macro	treat all classes equally
weighted	weight by support
micro	global TP/FP/FN (= accuracy if labels exclusive)

```
f1_score(y_te, y_hat, average="macro")
roc_auc_score(y_te, proba, multi_class="ovr", average="macro")
```

Metric picker

- **Balanced binary** → Accuracy + ROC-AUC
- **Imbalanced binary** → PR-AUC + F1 (at chosen threshold)
- **Need probabilities** → Brier + Log-loss (calibration!)
- **Asymmetric costs** → custom utility = TP·v – FP·c
- **Multi-class** → macro-F1 + Cohen's κ + confusion matrix

● Cross-validation

StratifiedKFold (default)

```
from sklearn.model_selection import StratifiedKFold,
cross_val_score
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X_tr, y_tr, cv=skf,
```

```
scoring="roc_auc", n_jobs=-1)
print(scores.mean(), scores.std())
Always stratified for classification. Preserves class ratio in each fold.
StratifiedGroupKFold
from sklearn.model_selection import StratifiedGroupKFold
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True,
random_state=42)
# groups = patient_id (rows from same patient go to same fold)
Scoring keys
```

Scoring	For
"accuracy"	balanced classes
"roc_auc"	binary, rank quality
"average_precision"	imbalanced — PR-AUC
"f1" / "f1_macro"	at fixed threshold
"neg_log_loss"	probability quality
"neg_brier_score"	calibration

▲ Hyperparameter tuning

GridSearchCV (≤3 hp)

```
grid = {"model_C": [0.01, 0.1, 1, 10, 100],
        "model_penalty": ["l1", "l2"],
        "model_solver": ["saga"]} # saga supports
both
gs = GridSearchCV(pipe, grid, cv=skf, scoring="roc_auc",
n_jobs=-1)
```

Optuna TPE (≥3 hp)

```
import optuna
def objective(trial):
    params = {
        "n_estimators": 2000,
        "learning_rate": trial.suggest_float("lr", 1e-3, 0.3,
log=True),
        "num_leaves": trial.suggest_int("leaves", 15, 255,
log=True),
        "min_child_samples": trial.suggest_int("mcs", 5, 100),
        "reg_lambda": trial.suggest_float("rl", 1e-3, 10,
log=True),
        "subsample": trial.suggest_float("sub", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("col", 0.5,
1.0),
    }
    model = lgb.LGBMClassifier(**params,
class_weight="balanced", random_state=42)
    return cross_val_score(model, X_tr, y_tr, cv=5,
scoring="roc_auc", n_jobs=-1).mean()
study = optuna.create_study(direction="maximize",
sampler=optuna.samplers.TPESampler(seed=42),
pruner=optuna.pruners.MedianPruner())
study.optimize(objective, n_trials=100)
```

Threshold tuning (CRITICAL for imbalanced)

```
from sklearn.metrics import precision_recall_curve
p_va = pipe.predict_proba(X_va)[: , 1]
prec, rec, thr = precision_recall_curve(y_va, p_va)
f1 = 2 * prec * rec / (prec + rec + 1e-12)
best_t = thr[np.argmax(f1[:-1])]
print("best threshold =", best_t)
y_hat_te = (pipe.predict_proba(X_te)[: , 1] > best_t).astype(int)
Tune threshold on VAL set, NEVER on test. Threshold is a hyperparameter.
```

◆ Calibration + diagnostics

Why calibrate?

RF / SVM / GBM produce scores in [0,1] that don't equal true P(y=1). If you'll use probabilities (Bayesian decisions, expected value), calibrate.

```
from sklearn.calibration import CalibratedClassifierCV,
calibration_curve
# Method "isotonic" (non-parametric, needs >1k samples) or
"sigmoid" (Platt)
cal = CalibratedClassifierCV(estimator=base_model,
method="isotonic", cv=5)
cal.fit(X_tr, y_tr)
# Diagnose:
```

```
prob_true, prob_pred = calibration_curve(y_te, p, n_bins=10)
plt.plot(prob_pred, prob_true, marker="o"); plt.plot([0,1],
[0,1], "k--")
Confusion matrix + classification_report
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(y_te, y_hat,
normalize="true")
# Per-class precision / recall / f1
print(classification_report(y_te, y_hat, digits=3))
Error analysis
```

- **Slice by feature:** does error concentrate in one region/segment?
- **Worst-K mistakes:** sort by |p–y|; inspect features
- **Confidence ranking:** are high-confidence errors structural (label noise)?
- **SHAP on misclassifieds:** which features pushed the model wrong?

Class explanation (SHAP)

```
import shap
expl = shap.TreeExplainer(model)
sample = X_tr.sample(500, random_state=42)
sv = expl(sample) # new API returns
Explanation object
shap.plots.beeswarm(sv) # multi-class:
sv[... , k]
shap.plots.bar(sv.abs.mean(axis=0)) # global feature
importance
shap.plots.waterfall(sv[0]) # single row, single
class
```

◆ Deployment

Joblib bundle (canonical)

```
import joblib
from datetime import datetime, timezone
bundle = {"pipeline": gs.best_estimator_,
        "threshold": best_t,
        "positive_label": 1, # explicit – avoid [:,
1] silent bug
        "feature_cols": list(X_tr.columns),
        "trained_at": datetime.now(timezone.utc).isoformat()}
joblib.dump(bundle, "model.joblib", compress=3)
# Load:
b = joblib.load("model.joblib")
y_hat = (b["pipeline"].predict_proba(X_new)[: , 1] >
b["threshold"]).astype(int)
```

ONNX (cross-runtime)

```
from skl2onnx import to_onnx
final_estimator = gs.best_estimator_.named_steps["model"]
onx = to_onnx(gs.best_estimator_, X_tr[:1].astype(np.float32),
options={id(final_estimator): {"zipmap":
False}}) # raw scores
# Serve via ONNXRuntime (10-50× faster inference, language-
agnostic)
```

FastAPI service

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
class Req(BaseModel): age: float; income: float; region: str
B = joblib.load("model.joblib")
PIPE = B["pipeline"]; POS_IDX =
list(PIPE.classes_).index(B["positive_label"])
@app.post("/predict")
def predict(r: Req):
    X = pd.DataFrame([r.dict()])[B["feature_cols"]] # enforce
column order
    p = float(PIPE.predict_proba(X)[0, POS_IDX])
    return {"proba": p, "label": int(p > B["threshold"])}
```

Drift monitoring

- **Input drift:** PSI / KS per feature vs train
- **Score drift:** histogram of P(y=1) over time
- **Label drift:** when labels arrive, monitor rolling base-rate
- **Performance drift:** rolling ROC-AUC / PR-AUC
- **Re-train:** PSI > 0.25 OR perf drop > 5%

▲ **Top traps (6-month memory)**

- **Accuracy on imbalanced data** — 99% accuracy on 1% positives = "predict 0 always". Use PR-AUC / F1.
- **Tuning threshold on test** — threshold is a hyperparameter; tune on VAL.
- **Target encoding outside CV** — fits on full data → leakage. Use sklearn 1.3+ TargetEncoder with cv.
- **Scaler fit on all data** — fits on test set rows; use Pipeline.
- **SMOTE before splitting** — synthetic positives leak into test. Resample INSIDE CV via imblearn.Pipeline.
- **Forgetting calibration** — RF/SVM/GBM scores ≠ probabilities. Calibrate if you use them as P(y=1).

- **Train-serve schema mismatch** — pin column order + dtypes in deployment bundle.
- **Single-model feature importance** — different models disagree. Combine SHAP + permutation.

◆ **Quick-reference (recall in 5 seconds)**

Task	Canonical pattern
Split	<code>train_test_split(test_size=0.2, stratify=y, random_state=42)</code>
Pipeline	<code>Pipeline([prep, LogisticRegression(class_weight="balanced")])</code>
Baseline	DummyClassifier + LogisticRegression

CV	<code>StratifiedKFold(5, shuffle=True, random_state=42)</code>
Tune (small)	<code>GridSearchCV(pipe, grid, cv=skf, scoring="roc_auc")</code>
Best model (tabular)	LightGBM + early stopping + <code>class_weight="balanced"</code>
Calibrate	<code>CalibratedClassifierCV(method="isotonic", cv=5)</code>
Threshold	<code>argmax(F1)</code> on val via <code>precision_recall_curve</code>
Imbalanced	<code>class_weight="balanced"</code> OR <code>scale_pos_weight</code> OR SMOTE in <code>imblearn.Pipeline</code>
Persist	<code>bundle {pipeline, threshold, columns}</code> via <code>joblib.dump</code>